Compiling and executing bundles

ar086 · 14 August 2026 · Draft

Contents

Developer guide

Compilation

`snn.compile` validates a network, validates an optional training recipe, reports target capabilities, and returns a `Bundle`. A target asks for diagnostics but never changes the graph.

```
bundle = snn.compile(net, target="tools/snn")
root = bundle.write("network.bundle", visualise=True)
```

A bundle contains canonical `graph.json`, optional `training.json`, a digest-bearing `manifest.json`, copied logical assets, a readable summary, and optional diagrams. Loading verifies hashes and rejects missing or altered files.

Dataset paths, output directories, accelerator caches, and experiment-specific analysis do not belong in the bundle.

Execution requests

`ExecutionSpec` selects the legacy or graph executor explicitly and supplies inputs, seed, device, recording profile, checkpoint, runtime state, and execution options.

The graph executor plans the complete topology before stepping. It validates dimensions, scheduling, polarity, delays, supplied inputs, outputs, and backend capabilities. Historical CLI commands continue to select the legacy executor by default.

Recordings and results

Recording profiles are:

- `full`, retaining every supported population trace;
- `observables`, retaining only explicitly exposed signals; and
- `none`, minimizing recording overhead.

Results contain named outputs, recordings, parameters, final voltages, runtime state, timing, device, and recording metadata.

Forward graph execution is implemented for dense COBA-LIF and leaky-integrator graphs with AMPA and GABA projections. Unsupported capabilities fail explicitly before execution.

API reference

Compile and load

```
snn.compile(network, *, training=None, target=None, assets=None) -> Bundle
snn.load_bundle(path) -> Bundle
snn.validate_graph(graph) -> ValidationResult
```

`target="tools/snn"` adds capability diagnostics without rewriting the graph. `assets` maps declared logical asset identifiers to physical paths. `load_bundle` verifies every manifest entry and graph digest before returning data.

`Bundle`

```
bundle.write(path, *, visualise=False) -> Path
bundle.visualise(path, *, view="circuit", scale=1) -> Path
```

Bundle fields are `graph`, `training`, `manifest`, `diagnostics`, and `asset_sources`. Visualization views are `"circuit"`, `"training"`, and `"expanded"`.

`ExecutionSpec`

```
ExecutionSpec(
    kind,
    executor="legacy",
    bundle=None,
    graph=None,
    inputs={},
    input_bindings=(),
    protocol={},
    seed=0,
    device="auto",
    recording="full",
    checkpoint=None,
    runtime_state=None,
    options={},
)
```

`kind` is `"build"`, `"simulate"`, `"train"`, or `"infer"`. `executor` is `"legacy"` or `"graph"`. Recording profiles are `"full"`, `"observables"`, and `"none"`. Device values are `"auto"`, `"cpu"`, `"cuda"`, `"cuda:N"`, or `"mps"`.

Dispatch and results

```
build(spec) -> ExecutionResult
simulate(spec, *, runtime_state=None) -> ExecutionResult
train(spec) -> ExecutionResult
infer(spec) -> ExecutionResult
execute_request(spec, *, legacy=None) -> ExecutionResult
```

`ExecutionResult` exposes `executor`, `outputs`, `recordings`, `parameters`, `final_state`, `runtime_state`, `metrics`, and `model`. Graph-native `train` is not implemented; it fails with the missing training capability instead of using the legacy trainer silently.

Next: Inputs, outputs, and readouts